Electrónica Digital 2: Laboratorio 3
Lógica secuencial y máquinas de estados finitos

[Nombre y apellido] [Nombre y apellido] [Nombre y apellido]
Ingeniería en Mecatrónica, Universidad Tecnológica (UTEC)
Fray Bentos, Uruguay
Correo: pendiente

***Resumen*—[Completar: Redactar al finalizar: objetivo del laboratorio; divisor de 100 MHz a un pulso por segundo; contador de diez bits; FSM de cuatro operaciones; método de validación; resultados efectivamente obtenidos y principal limitación. No atribuir éxito a pruebas pendientes.]**

***Index Terms*—FPGA, VHDL, lógica secuencial, divisor de frecuencia, contador, máquina de estados finitos, Moore, datapath, Vivado.**

I. Introducción

En el Laboratorio 3 de Electrónica Digital 2 se abordó el diseño de circuitos síncronos mediante VHDL sobre una FPGA Spartan-7, conforme a la consigna de la asignatura [1]. Se plantearon dos desarrollos relacionados: en la Parte A, un divisor de frecuencia y un contador binario de diez bits; en la Parte B, una máquina de estados finitos (FSM) encargada de secuenciar operaciones lógicas y aritméticas. La habilitación temporal diseñada para la primera parte se reutilizó conceptualmente en la segunda.

La validación se dividió en simulación funcional e implementación física. **[Completar: Precisar aquí, en pasado, cuáles etapas se completaron realmente y cuáles quedaron sin ejecutar.]**

I-A. Objetivos

Se establecieron los siguientes objetivos:

- Describir procesos síncronos en VHDL mediante `rising_edge` y reset.
- Calcular e implementar un divisor basado en un contador módulo `MAX`.
- Implementar un contador ascendente con habilitación *clock enable*.
- Describir una FSM de Moore mediante estados enumerados y tres procesos.
- Integrar la FSM como unidad de control y un *datapath* combinacional.
- Contrastar las formas de onda simuladas con el comportamiento de la FPGA.

II. Marco teórico

II-A. Lógica combinacional y secuencial

En un circuito combinacional, las salidas dependen únicamente de las entradas actuales. En uno secuencial, interviene además el estado interno, que conserva información de los ciclos anteriores [4]. En VHDL, un registro se describe mediante `rising_edge(clk)`; el reset síncrono se evalúa dentro de ese flanco, a diferencia del reset asíncrono, que puede actuar sin esperar al reloj [5].

II-B. División de frecuencia y habilitación

Un divisor por módulo se obtuvo mediante una cuenta interna y la detección del valor terminal (*terminal count*) [3]. Para la práctica se especificó un pulso `en` de un ciclo de duración cada `MAX` ciclos del reloj principal. El número de ciclos requerido se calculó como

$$\mathrm{MAX} = \frac{f_{\mathrm{clk}}}{f_{\mathrm{pulso}}}. \tag{1}$$

De este modo, los módulos conservaron el dominio de reloj de 100 MHz y utilizaron `en` para determinar cuándo actualizar su estado. El uso de un único dominio de reloj evitó utilizar una salida lógica como reloj de otro bloque, práctica desaconsejada en FPGA por las dificultades de ruteo y análisis temporal [3]. Este pulso de habilitación no fue una señal cuadrada de 1 Hz: su período fue de un segundo y su duración en alto fue de un único ciclo de 10 ns.

II-C. FSM de Moore y datapath

Una FSM está formada por un conjunto finito de estados, reglas de transición y salidas [4]. En una máquina de Moore, las salidas dependen del estado registrado; en una de Mealy, pueden depender del estado y de las entradas actuales. La enseñanza del curso propuso un patrón de tres procesos: registro de estado síncrono, próximo estado combinacional y salida combinacional [4]. En la Parte B se especificó la codificación Moore `op` correspondiente a cuatro estados; el *datapath*, de naturaleza combinacional, recibió A, B y `op`. Por tanto, la salida `op` depende solo del estado, mientras que el resultado de cálculo también depende de A y B.

III. Metodología

III-A. Materiales y herramientas

Se dispuso como plataforma de referencia de la Spartan Edge Accelerator Board con FPGA Xilinx Spartan-7 XC7S15, el shield multifunción UTEC, el cable USB-C, el programador JTAG HS2 y Vivado, versión indicada 2025.1 [1]. **[Completar: Confirmar modelo de programador, versión efectiva del software, dispositivo exacto y componentes realmente utilizados.]**

III-B. *Parte A: divisor de frecuencia*

Se diseñó la interfaz del módulo `divisor` con entradas `clk` y `rst`, salida `en` y genérico `MAX`. Se especificó un contador interno que se reiniciaba al alcanzar $\mathrm{MAX} - 1$ y activaba `en` durante un único ciclo. Para un reloj de 100 MHz y un pulso por segundo, se obtuvo

$$\mathrm{MAX}_{1\,\mathrm{Hz}} = \frac{100 \times 10^6}{1} = 100\,000\,000. \tag{2}$$

Para obtener 1 kHz, el valor correspondiente fue $\mathrm{MAX}_{1\,\mathrm{kHz}} = 100\,000$. El contador interno del divisor de 1 Hz requirió $\lceil \log_2(100\,000\,000) \rceil = 27$ bits. El valor terminal se estableció en $\mathrm{MAX} - 1$, no en MAX, para evitar un ciclo adicional.

III-C. *Parte A: contador e integración*

Se definió un contador ascendente con genérico `N=10`, reset síncrono y habilitación `en`. Su salida `q(9 downto 0)` se asignó a los diez LEDs, con `LED0` como bit menos significativo. En el módulo superior, la salida `en` del divisor se conectó a la entrada homónima del contador. Así, el registro solamente incrementaba cuando recibía un pulso de habilitación. El rango de diez bits fue de 0 a 1023, seguido de desborde a cero.

III-D. *Comportamiento temporal esperado*

El pulso `en` fue especificado como una habilitación de un único ciclo de reloj cada `MAX` ciclos. Con `MAX=4`, la separación entre dos activaciones consecutivas fue de cuatro períodos de reloj. Con `MAX=100 000 000` y reloj de 100 MHz, el intervalo fue de un segundo. En un contador binario, cada bit dividió por dos la frecuencia de la señal cuadrada representada por el bit inmediato inferior [3]. El avance de la cuenta permaneció sujeto a `en`, no a un flanco de una señal generada por lógica combinacional.

III-E. *Parte B: FSM y diagrama de estados*

Se definieron los estados enumerados `S_AND`, `S_OR`, `S_XOR` y `S_ADD`. Cada pulso de habilitación provocó, por especificación, una transición al siguiente estado y, después de `S_ADD`, el retorno a `S_AND`. El reset se definió con destino `S_AND`. Se separaron el registro de estado, la lógica combinacional del próximo estado y la decodificación de la salida `op`, siguiendo el patrón de tres procesos presentado en el material del curso [4]. La habilitación se consideró dentro de la lógica de transición o del registro, sin generar un segundo reloj.

III-F. *Parte B: datapath e integración*

El datapath recibió `A(3:0)`, `B(3:0)` y `op(1:0)`. Se definieron las operaciones y la visualización indicadas en la Tabla I. La suma se representó en cinco bits para conservar el acarreo, mientras que las operaciones lógicas se expresaron en cuatro bits. El divisor de la Parte A fue incorporado como fuente de habilitación para la FSM.

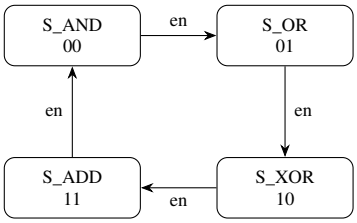


Figura 1. Diagrama de transición especificado para la FSM. Debe verificarse su correspondencia con el código final.

Tabla I
CODIFICACIÓN Y SALIDAS DE LA PARTE B

op	Estado	Cálculo	Salidas
00	AND	$A \wedge B$	LED3:0
01	OR	$A \vee B$	LED3:0
10	XOR	$A \oplus B$	LED3:0
11	ADD	$A + B$	LED4:0

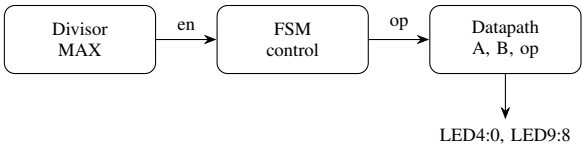


Figura 2. Diagrama de bloques funcional para la integración de la Parte B. Se omitieron las conexiones comunes de reloj y reset para facilitar la lectura.

III-G. *Simulación y testbenches*

Para la Parte A se planteó un escenario de prueba con `MAX=4`, que permitió observar los pulsos sin simular cien millones de ciclos entre habilitaciones. Se fijaron como criterios de verificación la duración de un ciclo de `en`, su periodicidad, el incremento unitario, el desborde y el reset. En el material de clase se recomendó medir períodos entre eventos con `now` y utilizar `assert`; al comparar valores después de un flanco, se propuso un retardo breve para permitir la actualización de los ciclos delta del simulador [3]. Para la Parte B se establecieron dos recorridos completos de la secuencia, un reset desde un estado distinto de `S_AND` y casos de prueba para cada operación, incluida una suma con acarreo. El material de FSM mostró la comparación de salidas esperadas mediante `assert`, práctica aplicable a la secuencia de `op` [4]. **[Completar: Indicar cuáles verificaciones se implementaron efectivamente con assert y cuáles se inspeccionaron visualmente.]**

III-H. *Restricciones e implementación en la placa*

La consigna estableció el reloj de 100 MHz en el pin H4 y requirió la restricción temporal `create_clock`. Las entradas y salidas del shield utilizaron el estándar `LVCMOS33`. Además, la salida `AR_3V3_EN` quedó definida como activa en bajo, con asignación al pin L13 y valor lógico cero para habilitar la alimentación de los dip-switches [1]. Las asignaciones físicas restantes se documentaron en el archivo XDC, de acuerdo con el manual del shield. **[Completar: Completar pines concretos del XDC empleado y describir**

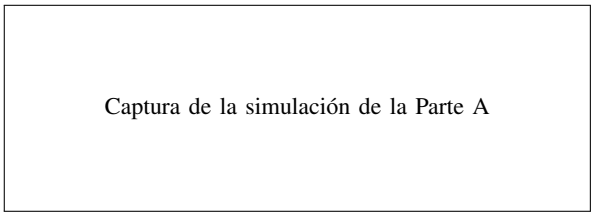


Figura 3. Verificación temporal del divisor y el contador. Imagen pendiente de incorporación.

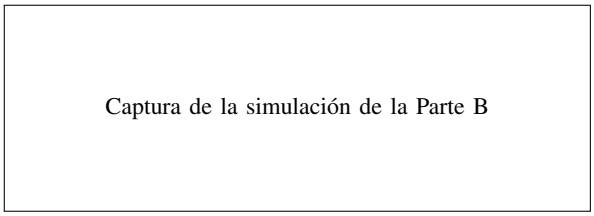


Figura 4. Transiciones de la FSM y salidas del datapath. Imagen pendiente de incorporación.

Tabla II
REGISTRO DE PRUEBAS DE LA FSM Y DEL DATAPATH

A	B	op	Esperado	Observado
0101	0011	00	00001	–
0101	0011	01	00111	–
0101	0011	10	00110	–
0101	0011	11	01000	–
1111	0011	11	10010	–

en pasado únicamente síntesis, implementación, bitstream y programación realmente realizados.]

IV. RESULTADOS Y DISCUSIÓN

IV-A. Parte A: simulación

[Completar: Insertar la captura real: clk, rst, en y q, con escala de tiempo legible. Indicar MAX empleado, instantes de pulso, comparación con el modelo y resultado de assert si existió.]

IV-B. Parte A: funcionamiento físico

[Completar: Registrar los valores observados durante al menos diez segundos, la conmutación de LED0, el incremento cada segundo y el comportamiento del reset. Si no se implementó, indicar expresamente la limitación.]

IV-C. Parte B: simulación

[Completar: Insertar captura de al menos dos ciclos de la FSM; identificar clk, rst, en, op, A, B, resultado y carry. Analizar reset y sumas con acarreo.]

IV-D. Parte B: validación física

[Completar: Describir valores A y B usados, secuencia visible en LED9:8, resultado en LED3:0, carry en LED4 y reset. Comparar lo esperado con lo observado, sin marcar como verificados los casos aún no ensayados.]

La Tabla II presenta casos de referencia calculados, no mediciones. El campo observado quedó sin completar hasta su contrastación experimental. Los cinco bits de salida se representaron como `LED4:0`.

IV-E. Discusión y limitaciones

[Completar: Analizar diferencias entre simulación y placa, problemas concretos de restricciones, síntesis, lectura de switches o temporización; explicar sus causas y las soluciones efectivamente aplicadas. Valorar la legibilidad del video e indicar si las tomas permitieron verificar las entradas y salidas.]

V. PREGUNTAS DEL LABORATORIO

V-A. Parte A

V-A1. Circuito combinacional, secuencial y proceso síncrono: La distinción principal fue la presencia de memoria: un circuito combinacional entregó una función de las entradas presentes y uno secuencial conservó un estado entre ciclos. En VHDL, `rising_edge(clk)` delimitó el instante de actualización de los registros. Con reset síncrono, su acción quedó sujeta al mismo flanco, lo que hizo posible un comportamiento temporal determinista.

V-A2. MAX para 1 Hz y 1 kHz y anchura del contador: Aplicando la Ec. (1), se obtuvo $\mathrm{MAX}_{1\,\mathrm{Hz}} = 100\,000\,000$ y $\mathrm{MAX}_{1\,\mathrm{kHz}} = 100\,000$. Para representar los valores comprendidos entre cero y 99 999 999, fueron suficientes 27 bits porque $2^{26} = 67\,108\,864 < 100\,000\,000 \leq 2^{27} = 134\,217\,728$.

V-A3. Habilitación en lugar de reloj dividido: El empleo de `clock enable` mantuvo el contador y el divisor sincronizados con el mismo reloj físico. Así se evitó tratar una señal generada por lógica ordinaria como una nueva red de reloj, lo que habría complicado la distribución, las restricciones temporales y el análisis de cruces entre dominios. La operación de un registro quedó condicionada a `en='1'` sin alterar su fuente de reloj.

V-A4. Uso de genéricos en simulación: Con el valor real de cien millones de ciclos por pulso, una simulación de varios incrementos implicaría una cantidad muy elevada de eventos, tal como se explicó en el caso de estudio del material de divisores [3]. El parámetro genérico `MAX` permitió sustituir el valor de implementación por uno pequeño, como cuatro, sin modificar la estructura lógica. **[Completar: Confirmar el MAX realmente utilizado y la duración de la prueba.]**

V-A5. Frecuencia de conmutación de los LEDs: Para incrementos a 1 Hz, el bit i del contador cambió de nivel cada 2^i segundos, y completó un período de encendido y apagado cada 2^{i+1} segundos. En consecuencia, su frecuencia de señal cuadrada fue

$$f_{\mathrm{LED}i} = \frac{1\,\mathrm{Hz}}{2^{i+1}}. \tag{3}$$

Por ejemplo, `LED0` tuvo una frecuencia de señal de 0,5 Hz (una transición por segundo), `LED1` de 0,25 Hz y `LED9` de $1/1024$ Hz. Esta distinción evitó confundir la frecuencia de incremento del contador con la frecuencia de parpadeo de cada LED.

V-B. Parte B

V-B1. Definición y componentes de la FSM: La FSM se representó mediante un registro de estado, una lógica de transición y una función de salida. Los cuatro estados de la Fig. 1 representaron, en orden, AND, OR, XOR y ADD. Cada habilitación especificó el paso siguiente; después de ADD se regresó a AND. El reset estableció el estado inicial AND.

V-B2. Diferencia entre Moore y Mealy: En una máquina de Moore, la salida codificada depende exclusivamente del estado almacenado; en una máquina de Mealy, también puede depender de las entradas actuales. La FSM propuesta fue de Moore porque `op` se obtuvo del estado. Aunque los resultados del datapath variaron combinacionalmente al modificarse A o B, ello no convirtió a la FSM de control en una máquina de Mealy.

V-B3. Estados enumerados y síntesis: La descripción mediante un tipo enumerado permitió utilizar nombres semánticos y redujo errores asociados a códigos binarios asignados manualmente. En general, la síntesis transforma los estados simbólicos a una representación física apropiada y puede escoger una codificación distinta de la codificación de salida `op`, según sus opciones de optimización. Por tanto, la numeración de `op` no implicó necesariamente que el registro físico de estado utilizara exactamente esos dos bits.

V-B4. Separación de procesos síncronos y combinacionales: El registro de estado se planteó en un proceso síncrono para conservar el estado entre flancos y actualizarlo solamente al recibirse una habilitación. La lógica de próximo estado se planteó combinacionalmente para evaluar las reglas de transición en función del estado actual. La salida de Moore se definió como función combinacional de ese estado. **[Completar: Citar un fragmento breve del código real que demuestre esta separación.]**

V-B5. Relación con la arquitectura de un procesador: La FSM cumplió una función de control al seleccionar la operación mediante `op`. El datapath realizó las transformaciones de datos sobre A y B y generó los resultados. Esta separación fue análoga, en una escala sencilla, a la organización de un procesador: la unidad de control determina la acción y la ruta de datos la ejecuta. La arquitectura del laboratorio no constituyó un procesador completo, ya que no incorporó un programa de instrucciones ni todos sus registros y rutas de acceso.

VI. Conclusiones

La solución propuesta mostró una relación funcional directa entre las dos partes: la habilitación periódica requerida para incrementar un contador también permitió regular las transiciones de una FSM sin agregar un dominio de reloj. Esta decisión de diseño fue coherente con las recomendaciones del material del curso sobre distribución de reloj en FPGA [3].

La separación del control y del datapath permitió distinguir el estado de la operación de los valores procesados. Asimismo, el modelo de Moore hizo posible interpretar el código `op` sin depender de los operandos, aunque el resultado combinacional sí dependiera de ellos [4].

[Completar: Agregar análisis experimental: contrastar periodicidad, reset, dos ciclos de FSM y resultados del datapath usando evidencia propia; identificar discrepancias, causas y correcciones reales. Sin estos datos no es posible concluir que el hardware funcionó según lo previsto.]

[Completar: Añadir una mejora específica fundamentada en una dificultad observada. Evaluar la calidad del video: encuadre solamente de la placa, iluminación, switches y LEDs legibles, ritmo pausado y explicación de las pruebas.]

Evidencias y archivos

[Completar: Enlazar el repositorio o adjuntar los códigos divisor, contador, top A, FSM, datapath, top B, ambos testbenches y los archivos XDC; añadir el enlace de un video por parte y las capturas de simulación. Para el video, utilizar tomas pausadas y encuadrar únicamente la placa, con LEDs y dip-switches legibles.]

Referencias

[1] Universidad Tecnológica (UTEC), "Laboratorio 3: Lógica secuencial y máquinas de estados finitos, Partes A y B," Electrónica Digital 2, guía de laboratorio, 2026.

[2] Universidad Tecnológica (UTEC), "Rúbrica de evaluación por niveles de logro: Laboratorio 3," Electrónica Digital 2, 2026.

[3] Universidad Tecnológica (UTEC), *Todo divisor de frecuencia es, en el fondo, un contador*, material didáctico de Electrónica Digital 2, s. f.

[4] Universidad Tecnológica (UTEC), *Sistemas que se comportan distinto según su historia*, material didáctico de Electrónica Digital 2, s. f.

[5] AMD, *Vivado Design Suite User Guide: Synthesis (UG901)*. [En línea]. Disponible: https://docs.amd.com/r/en-US/ug901-vivado-synthesis

[6] AMD, *Vivado Design Suite User Guide: Logic Simulation (UG900)*. [En línea]. Disponible: https://docs.amd.com/r/en-US/ug900-vivado-logic-simulation

[7] AMD, *Vivado Design Suite User Guide: Using Constraints (UG903)*. [En línea]. Disponible: https://docs.amd.com/r/en-US/ug903-vivado-using-constraints